

Lab Overview

About This Lab

Welcome to **Building and Deploying Digital Twin Applications With Omniverse Kit App Streaming**. In this hands-on lab, we'll walk through the complete lifecycle of creating a streamable NVIDIA Omniverse Kit application and deploying it to the cloud for production use.

Kit App Streaming is the technology that powers real-world digital twin deployments at scale. Companies use this platform to stream GPU-rendered 3D applications to users through their web browsers, eliminating the need for powerful local hardware. By the end of this lab, you'll have firsthand experience with the same tools and workflows used in production environments.

We'll start by building a Kit application from a template, connect it to a web client for local development and testing, then package it into a Docker container and deploy it to an NVIDIA Cloud Functions (NVCF) cluster. This is the real stack – the same infrastructure that NVIDIA and its partners use in production.

What Is Kit App Streaming?

At the most fundamental level, Kit App Streaming means streaming GPU-rendered pixels from a Kit application to a web client, while allowing the two to communicate. This is a **peer-to-peer solution**: one Kit app streamed to one client.

Local Streaming

For iterating and proof-of-concept work, the entire deployment can run locally on a single workstation. The Kit app, signaling server, media server, and web client all run on the same machine. This is where development happens.

Cloud Streaming

For scalable production solutions, an API launches Kit applications on demand in the cloud. In this lab, we use **NVIDIA Cloud Functions (NVCF)** – a managed deployment platform that offloads hardware, streaming, and network complexities. NVIDIA also offers a self-managed option called Kit App Streaming Self-Hosted for teams that need full control over their infrastructure.

The Digital Twin Context

To build a comprehensive operational digital twin, three essential components come together:

1. **Physically accurate 3D models with OpenUSD** – Detailed models of industrial systems and environments serve as the visual foundation for digital twins. OpenUSD (Universal Scene Description) is the open-source framework that makes this possible.
2. **Real-time streaming with Omniverse** – Kit App Streaming delivers these interactive 3D environments to web browsers, enabling remote visualization and collaboration without powerful local hardware.

3. **Real-time data analysis and visualization** – Integrating live IoT data with the 3D environment creates dynamic dashboards that provide actionable insights alongside the visual twin.

Note

This lab focuses on the **streaming** component – building, deploying, and connecting Kit applications. The follow-on lab, *Develop AI-Powered Operational Dashboards for Digital Twin Applications*, picks up where we leave off and covers integrating custom data and bi-directional messaging between the Kit app and web client.

Who Builds These Solutions?

In production, a Kit App Streaming team typically includes **web developers** (building the client UI), **Python and/or C++ developers** (building Kit extensions and application logic), and **DevOps engineers** (managing infrastructure and deployments). For this lab, we'll focus on the web development and deployment workflows. The infrastructure has already been set up for you.

Learning Objectives

By the end of this lab, you'll be able to:

1. **Create** a USD Viewer Kit application using the Kit App Template
2. **Configure** a web client to stream a Kit application locally using direct WebRTC streaming
3. **Package** a Kit application into a Docker container for cloud deployment
4. **Deploy** a containerized Kit application to an NVCF cluster using the NVCF CLI
5. **Verify** end-to-end cloud streaming by connecting a web client to a deployed Kit application

What You'll Build

This lab follows a real-world development workflow through three major stages:

Stage 1: Build and Stream Locally

Build a Kit application from a template and stream it to a web client running on your machine. This is the **development phase** where you iterate on your application and web UI together.

Stage 2: Containerize and Deploy

Package your Kit application into a Docker container and deploy it to an NVCF cloud cluster. This is the **deployment phase** where you prepare your application for production.

Stage 3: Stream from the Cloud

Connect a new web client to your cloud-deployed Kit application and verify the stream. This is the **production phase** where your application is running at scale.

Prerequisites

This lab assumes no prior experience with Kit App Streaming, Docker, or Kubernetes. All tools and repositories are pre-configured in your lab environment. You should, however, have basic familiarity with:

- Using a **terminal** to run commands
- Navigating **file directories** from the command line
- Editing files in **VS Code** (or a similar code editor)
- Basic understanding of what a **web application** is

Note

All required repositories, tools, and dependencies are pre-installed in your lab environment. You do not need to install anything yourself.

Lab Structure

This lab is divided into the following sections:

Section	What You'll Do	Key Concepts
Environment Setup	Orient to your workspace and prepare your IDE	VS Code, terminals, workspace layout
Building Your Kit Application	Build a USD Viewer Kit app from template	Kit App Template, <code>repo.sh</code> , application layers
Local Streaming	Create a web client and stream your Kit app locally	WebRTC, signaling server, <code>npm</code> , direct streaming
Cloud Deployment	Containerize and deploy your Kit app to NVCF	Docker, NVCF CLI, function definitions, <code>kubectl</code>
Cloud Streaming	Connect a web client to your cloud-deployed app	NVCF endpoints, API keys, function IDs
Conclusion	Review what you've accomplished and explore next steps	Architecture recap, resources
Knowledge Check	Test your understanding of key concepts	Multiple-choice review

Tip

Throughout this lab, the instructor will pause at key milestones to ensure everyone is on the same page. If you need help at any point, display your **red cup** and a TA will come to you. Once your question is answered, the TA will switch it back to **blue**. If you finish a section early, take a moment

to explore the code and configuration files – understanding what’s happening under the hood will deepen your learning.

Environment Setup

Before we start building, let's take a moment to orient ourselves in the lab environment. Your workspace comes pre-configured with everything you need – repositories, tools, and dependencies are already installed.

Explore Your Workspace

Your lab workspace contains a pre-cloned repository and supporting lab files at the root level:

```
workspace/
├── kit-app-template/           # NVIDIA Kit App Template repository
├── create-function.json       # NVCF function definition (pre-configured)
├── deploy-function.json       # NVCF deployment specification (pre-configured)
├── function-info.txt          # Notepad for storing API keys and IDs
└── (supporting scripts)       # Preparation scripts for cloud deployment
```

Let's get familiar with what's here:

- **kit-app-template/** – This is the official NVIDIA Kit App Template repository. We'll use it to build our Kit application in the next section.
- **create-function.json** and **deploy-function.json** – Pre-configured files we'll use later when deploying to the cloud. You won't need to create these from scratch.
- **function-info.txt** – A scratchpad file where you'll save API keys and IDs generated during cloud deployment.
- **Supporting scripts** – Pre-configured scripts that set up tools needed for web development and cloud deployment. We'll use these as we progress through the lab.

Open a Terminal

We'll be working heavily in the terminal throughout this lab. Open a new terminal in VS Code:

- Navigate to **Terminal > New Terminal** (or press **Ctrl+Shift+`**)

Note

Throughout this lab, we'll use multiple terminal windows. VS Code lets you open several terminals side by side. When the instructions say "open a new terminal," click the + icon in the terminal panel or press **Ctrl+Shift+`**.

Important

Do not modify, delete, or rearrange files in the workspace before the lab begins. The pre-configured structure is essential for the lab to work correctly.

What's Next

Now that you're oriented in the workspace, we'll move on to building your first Kit application. In the next section, you'll use the Kit App Template to create a USD Viewer application and build it from source.

Building Your Kit Application

In this section, we'll create a Kit application using NVIDIA's Kit App Template. This template provides a tool that generates a complete, buildable application with all the configuration you need for streaming and deployment.

What Is a Kit Application?

A Kit application is a modular application built on NVIDIA's Omniverse Kit framework. Kit applications are composed of extensions – plug-in modules that add functionality like viewports, menus, and streaming capabilities. The Kit App Template gives you a starting point with sensible defaults so you can focus on your use case rather than boilerplate configuration.

Navigate to the Kit App Template

In your terminal, change into the `kit-app-template` directory:

- Navigate to the Kit App Template:

```
cd kit-app-template
```

This repository contains the template tool and build system we'll use to create our application.

Build a Kit Application From Template

Run the template tool to generate a new application:

- Launch the template tool:

```
./repo.sh template new
```

The tool will walk you through a series of prompts. Follow these selections:

- **Accept the governing terms** – Read and accept the license agreement
- **What would you like to create?** – Select **Application**
- **Which template would you like to use?** – Select **USD Viewer**
- **Application name and other defaults** – Press **Enter** to accept the defaults for each prompt (application name, version, etc.)
- **Extensions** – Accept the defaults (setup extension and messaging extension are pre-selected)
- **Would you like to add application layers?** – Select **Yes**
- **Select application layers** – Press **Space** to select both:
 - **Kit App Streaming** (for local streaming)
 - **NVCF Streaming** (for cloud deployment)

Note

Application layers are additional configuration files (`.kit` files) that customize how your application behaves in different environments. By adding the Kit App Streaming and NVCF layers now, we're preparing our application for both local development and cloud deployment later in the lab.

Tip

The USD Viewer template creates a lightweight application designed for viewing USD (Universal Scene Description) files. It's the ideal starting point for streaming use cases because it includes a viewport and basic navigation controls without unnecessary UI complexity.

After the prompts, the template tool generates your application's project structure under the `source/` directory, including:

- A main `.kit` file with all dependencies, extensions, and settings
- Simplified streaming `.kit` files for each application layer you selected

If you are missing your application layer streaming `.kit` files, run:

- `./repo.sh template modify` to select application layers again.

Build the Application

Now let's build the application. This pulls dependencies and compiles all the extensions:

- Build the application:
`./repo.sh build`

Important

The first build takes several minutes as it pulls packages from NVIDIA registries and compiles dependencies. This is normal. Subsequent builds are significantly faster because assets are cached locally.

- Verify the build completed successfully. Look for the **BUILD (RELEASE) SUCCEEDED** message in the terminal output.

Warning

If you don't see **BUILD (RELEASE) SUCCEEDED**, the build may have failed. Display your **red cup** and a TA will come assist you.

What You've Accomplished

You've just created a complete Kit application. Let's recap what happened:

1. The template tool generated a project structure with source code, configuration, and build files
2. You selected the **USD Viewer** template, which provides a streamable 3D viewport
3. You added **streaming layers** that configure the application for local and cloud streaming

4. The build system compiled everything into a runnable application

In the next section, we'll set up a web client and stream this application locally so you can see it in action.

Local Streaming

Now that we've built our Kit application, let's see it in action. In this section, we'll create a web client, launch the Kit app in streaming mode, and connect the two so you can view the 3D application right in your browser.

This is the **local development workflow** – you run your Kit app and web client side by side so you can iterate on both simultaneously.

How Local Streaming Works

When you stream a Kit application locally, two processes work together:

1. **The Kit application** runs on the GPU and renders 3D content. It sends its video output over WebRTC.
2. **The web client** runs in your browser and receives the WebRTC stream, displaying the rendered viewport.

The two communicate through a **signaling server** (for connection setup) and a **media server** (for the actual video stream). In local/direct mode, these servers run on your machine.

Create a Web Client for Direct Streaming

We'll use a separate terminal for the web client so we can keep the Kit application running in its own terminal later.

- **Open a new terminal** in VS Code (click the + icon in the terminal panel or press **Ctrl+Shift+`**)

Before running the web client template tool, configure npm to use the NVIDIA package registry:

- Set up the NVIDIA npm registry:

```
npm config set "@nvidia:registry"
"https://edge.urm.nvidia.com/artifactory/api/npm/omniverse-client-npm/" --
location=user
```

Now run the web client template tool to scaffold a streaming client:

- Run the web client template tool:

```
npx @nvidia/create-ov-web-rtc-app
```

The tool will prompt you with several options. Follow these selections:

- **Project name** – Type `direct-stream` and press **Enter**
- **Author** – Type NVIDIA (or your name) and press **Enter**
- **Project description** – Press **Enter** to accept the default
- **Output directory** – Press **Enter** to accept the default
- **Select UI framework** – Select **None TypeScript only**

- **Select stream source** – Select **Local direct connection**
- **Streaming library** – Press **Enter** to accept the default

Note

We're choosing **Local direct connection** because we want to stream directly from our locally running Kit app to the web client. Later, we'll create a second web client that connects through NVCF for cloud streaming.

Launch the Kit Application in Streaming Mode

Now let's start the Kit application. Switch back to the terminal where you built the application – you should already be in the `kit-app-template` directory.

- **Switch to the Kit App Template terminal** (the terminal you used to build the application)
- Launch the Kit application:

```
./repo.sh launch -- --no-window
--/app/auto_load_usd='/config/workspace/data/DataCenterDigitalTwin.usd'
```
- When prompted with launch options, select the **streaming** application layer:
`my_company.my_usd_viewer_streaming.kit`

This launches your Kit application in headless streaming mode – it renders on the GPU but doesn't open a visible desktop window. Instead, it waits for a WebRTC client to connect.

The `--/app/auto_load_usd` flag tells Kit to load a USD scene on startup. Here we're pointing it to a Data Center digital twin scene so you'll see a real 3D environment streaming in your browser rather than an empty viewport.

Note

The `--no-window` flag is important. Since we're running in a remote cloud environment (not on your local laptop), there's no display to show a window on. The application runs entirely in the background, streaming its output over WebRTC.

Tip

The first launch takes extra time as it pulls shader caches and initializes resources. If you see a warning like `failed to open active user not found`, you can safely ignore it – this is expected in the lab environment. Look for the `app ready` message to confirm the application has started successfully.

Leave this terminal running. The Kit application is now active and waiting for a streaming connection.

Configure the Web Client

Switch back to your **web client terminal** (the one where you ran the template tool) and follow these steps:

- Navigate into the web client project:

```
cd direct-stream
```

- Open the file `src/main.ts` (around line 76)
- Update the **signalingServer** address: Replace `localhost` with **your instance's IP address**
- Add a **mediaServer** parameter under the signaling server, also set to your instance's IP address:

```
mediaServer: '<YOUR_IP_ADDRESS>',
```

- Save the file

Tip

Where do I find my IP address? Look at the URL in your browser's address bar – the IP address is the part before the colon and port number. For example, if your URL is `http://18.220.82.185:8080`, your IP address is `18.220.82.185`.

Important

Make sure you don't include `http://` in the IP address fields. Enter only the IP address itself (e.g., `18.220.82.185`).

Install Dependencies and Start the Web Client

Now let's install the web client's dependencies and start it:

- Install dependencies:

```
npm install
```

Note

You may see some warnings during installation – these are safe to ignore. The most critical dependency pulled in by `npm install` is the **Omniverse WebRTC Streaming Library**. This is the library that enables a web client to present a streamed Kit application and provides **bi-directional messaging** between the client and the Kit app.

- Start the development server:

```
npm run dev -- --host 0.0.0.0
```

Important

Why --host 0.0.0.0? Your lab environment runs VS Code remotely in the cloud, not on your local laptop. By default, Vite (the web development server) only listens on `localhost`, which means you can't access it from your browser. Setting `--host 0.0.0.0` tells Vite to accept connections from any network interface, making it reachable from your browser.

Tip

VS Code may show an **“Open in Browser”** notification when the dev server starts. Ignore this popup – we'll use a different method to open the stream in the next step.

View the Stream

- **Open a new terminal** in VS Code (click the + icon in the terminal panel or press **Ctrl+Shift+`**) and run the following command to get the URL for your stream:

```
echo "http://$(curl icanhazip.com):5173"
```
- **Control+click** the URL to open it in a new browser tab

You should see the web client interface with a live view of your Kit application's viewport. You're now streaming your Kit application through WebRTC – try navigating the 3D scene using your mouse.

Warning

If you see **“Waiting for stream...”** indefinitely, verify that:

- Your Kit application is still running in the other terminal
- The signaling server and media server addresses in `src/main.ts` are correct (using your IP, not `localhost`)
- You don't have `http://` in the address fields

Stop the Local Processes

Before we move on to cloud deployment, let's stop the locally running processes. Go to each terminal where a process is running and press **Ctrl+C** to stop it:

- **Web client terminal** (`npm run dev`): Press **Ctrl+C**
- **Kit application terminal**: Press **Ctrl+C**

Both processes should terminate cleanly.

What You've Accomplished

Congratulations – you've just set up a complete local streaming pipeline. Here's what you had running:

- **Terminal 1:** Your Kit application, rendering 3D content on the GPU and streaming via WebRTC
- **Terminal 2:** A Vite development server hosting your web client
- **Browser:** Displaying the live stream from your Kit application

This is exactly how Kit App Streaming development works in practice: you iterate on both the Kit app and the web client simultaneously, testing changes in real time.

Pause Point

Wait here until the instructor signals to continue.

If you're stuck or don't see a working stream, display your **red cup** and a TA will come assist you. Once your issue is resolved, the TA will switch it back to **blue**.

Cloud Deployment

Now that you've seen your Kit application streaming locally, it's time to take it to the cloud. In this section, we'll package the application into a Docker container, push it to a container registry, and deploy it to an NVIDIA Cloud Functions (NVCF) cluster.

This is the transition from **development** to **production**. The same application you just streamed locally will soon be running on dedicated GPU hardware in a cloud environment.

The Deployment Pipeline

Here's what we'll do in this section, step by step:

1. **Package** the Kit application into a Docker container
2. **Push** the container to a container registry
3. **Set up** the NVCF CLI
4. **Generate** an API key
5. **Register** the application as an NVCF function
6. **Deploy** the function to a GPU node

Let's get started.

Build the Docker Container

Switch to the terminal where you previously built and ran the Kit application – you should already be in the `kit-app-template` directory.

- Navigate to the `kit-app-template` directory (if you're not already there):

```
cd kit-app-template
```
- Package your Kit application into a Docker container:

```
./repo.sh package_container --image-tag localhost:32000/kas/my-kit-app
```
- When prompted, select the **NVCF** version:

```
my_company.my_usd_viewer_nvcf.kit
```

Let's break this command down:

Part	What It Does
<code>./repo.sh package</code>	Runs the Kit App Template's packaging tool
<code>_container</code>	Tells it to create a Docker container (not just a zip archive)
<code>--image-tag</code>	Specifies the name and location for the container image

`localhost:32000/kas/my-kit-app`

The image tag – `localhost:32000` is the local container registry

Note

This build process takes several minutes. While it runs, the packaging tool pulls the Omniverse Ubuntu base image, bundles all your application’s dependencies, and creates a deployable container image.

- Once the build completes, verify that the container was created:

```
docker image list
```

You should see your newly created image in the output:

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
<code>localhost:32000/kas/my-kit-app</code>	<code>latest</code>	<code>ec3ce7d2a6d4</code>	<code>18 seconds ago</code>	<code>3.13GB</code>

Tip

Don’t worry if the size or image ID differ – what matters is that a row appears with `localhost:32000/kas/my-kit-app` and a recent timestamp.

Push the Container to the Registry

Now that the container is built and verified, push it to the local container registry:

- Push the container image:
`docker push localhost:32000/kas/my-kit-app`
- Wait for the push to complete. You’ll see progress bars for each layer being uploaded.

Note

In a production environment, you would push to a container registry like NVIDIA NGC, Docker Hub, or AWS ECR. In this lab, we’re using a local registry that’s part of your lab environment.

Upload the USD Stage to MinIO

Before we register our function, we need to upload the digital twin USD scene to an S3-compatible object store so the cloud-deployed Kit application can access it. Your lab environment includes a **MinIO** server that acts as a local S3-compatible object store running inside the cluster.

- **Open a new terminal** in VS Code (click the + icon in the terminal panel or press **Ctrl+Shift+`**)
- Run the preparation script:

```
bash ./prepare-kas.sh
```

This script configures your environment with the necessary credentials and endpoints for your NVCF cluster.

- Upload the USD scene data to the MinIO bucket:

```
aws s3 cp --recursive /config/workspace/data s3://kas/
```

This copies the Data Center digital twin scene and its dependencies to the `kas` bucket in MinIO. When the Kit application launches in the cloud, it will load the USD scene from this bucket using an internal cluster URL.

Note

MinIO is an S3-compatible object store. The AWS CLI in your lab environment is pre-configured to point to the local MinIO server, so standard `aws s3` commands work exactly as they would with real AWS S3.

Set Up the NVCF CLI

Now we need to configure the NVCF command-line interface (CLI) so we can register and deploy our application.

- Initialize the NVCF CLI:

```
nvcf init
```

This authenticates you with the cluster and sets up your admin token. You should see a confirmation message.

Tip

You can verify that the CLI is working by listing existing functions:

```
nvcf function list
```

If this is a fresh environment, you'll see **"No functions found"** – that's expected. We're about to create one.

Generate an API Key

Before deploying, generate an API key that your web client will use to authenticate with the NVCF cluster:

- Generate the API key:

```
nvcf api-key generate
```

The API key will be displayed in the terminal output. It starts with `nvapi-`.

- Open the `function-info.txt` file in your workspace and paste the **entire API key** into it
- Save the file

Warning

Save this API key immediately. Once the terminal output scrolls past, you cannot retrieve it again. If you lose the key, you can generate a new one by running `nvcf api-key generate` again. If you hit the maximum number of keys, delete an old one first:

```
nvcf api-key list
nvcf api-key delete --key-id <KEY_ID>
```

Create a Function Definition

NVCF organizes workloads as **functions**. A function definition tells the cluster what container to run, what resources it needs, and how to configure it.

Your workspace already contains a pre-configured function definition file called `create-function.json`. Let's review it:

- Open the file `create-function.json` in your workspace

The file defines your streaming function with settings for health checks, container configuration, and the USD scene to load. Key configuration values include:

- `--/app/livestream/nvcf/sessionResumeTimeoutSeconds=300` – Sets a 5-minute session resume timeout
- `--/app/auto_load_usd='http://minio.minio.svc.cluster.local:9000/kas/DataCenterDigitalTwin.usd'` – Tells the Kit application to load the digital twin scene from the MinIO bucket you just uploaded to
- Replace `<sceneUrl>` in `--/app/auto_load_usd=` with:
`http://minio.minio.svc.cluster.local:9000/kas/DataCenterDigitalTwin.usd`
- Save the file
- Register the function with the NVCF cluster:

```
nvcf function create --input-file create-function.json
```

When the command succeeds, it will output two critical pieces of information:

- **Function ID** – A unique identifier for your function
- **Version ID** – A unique identifier for this version of your function
- **Copy both IDs** to your `function-info.txt` file. You'll need them in the next step and again when configuring the cloud web client.

Tip

If you forget to record these IDs, you can always retrieve them by running:

```
nvcf function list
```

Deploy the Function

Your workspace also contains a pre-configured deployment specification file called `deploy-function.json`. We need to add the function and version IDs before deploying.

- Open the file `deploy-function.json` in your workspace
- Replace `<functionId>` with the **function ID** from the previous step
- Replace `<versionId>` with the **version ID** from the previous step
- Save the file
- Deploy the function:

```
nvcf function deploy create --input-file deploy-function.json
```

The deployment process will begin. This provisions a GPU node, pulls your container image, and starts your Kit application in the cloud.

Monitor the Deployment

While the function deploys, let's monitor its progress:

- Check the pod status:

```
kubectl get pods -n nvcf-backend
```

Tip

`kubectl` (pronounced “kube-control” or, among those in the know, “kube-cuddle”) is the Kubernetes command-line tool. It lets you inspect and manage the cluster. The `-n nvcf-backend` flag specifies the namespace where NVCF runs your functions.

Watch for a pod with your function name to appear. It will go through several states:

Status	Meaning
Pending	The cluster is provisioning resources
ContainerCreating	The container image is being pulled and started
Running (1/2)	The container is starting but not fully ready
Running (2/2)	The function is fully deployed and ready for streaming

When you see **2/2** under the **READY** column, your Kit application is running in the cloud and ready to accept streaming connections.

Note

The deployment typically takes 2-5 minutes. If the pod stays in **Pending** for more than 10 minutes, display your **red cup** to signal for help.

Troubleshooting

Note

If you deploy a misconfigured function, you can **Delete** the function and try again. Run: `nvcf function delete --function-id <id> --version-id <version>` with your **function ID** and **version ID**.

What You've Accomplished

You've taken your Kit application from a local development project to a cloud-deployed service:

1. **Packaged** the application into a Docker container
2. **Pushed** the container to a registry
3. **Registered** it as an NVCF function with a deployment specification
4. **Deployed** it to a GPU node in the cloud

Your Kit application is now running in an NVCF cluster, waiting for a web client to connect. In the next section, we'll create that client.

Pause Point

Wait here until the instructor signals to continue.

While you wait, you can explore the cluster by running `kubectl get pods -A` to see all the pods that make up the Kit App Streaming platform. This is the same infrastructure stack that NVIDIA runs in production for real customers.

If you're stuck or your pod isn't reaching **2/2 Ready**, display your **red cup** and a TA will come assist you. Once your issue is resolved, the TA will switch it back to **blue**.

Cloud Streaming

Your Kit application is now deployed and running on a GPU node in the NVCF cluster. In this final hands-on section, we'll create a new web client configured to connect to the cloud-deployed application and verify end-to-end streaming.

How Cloud Streaming Differs From Local

In the previous local streaming section, your web client connected directly to the Kit application using a signaling server and media server running on the same machine. In cloud streaming with NVCF, the architecture is different:

- **NVCF manages the connection infrastructure** – it handles the signaling, session management, and UDP routing for you
- **Your web client connects to the NVCF API endpoint** instead of a local server
- **Authentication is required** – your web client needs an API key and function ID to request a stream

This is more complex under the hood, but NVCF abstracts away the hard parts so you can focus on your application.

Create an NVCF Web Client

We'll use the same web client template tool from earlier, but this time we'll select the NVCF stream source.

- **Open a new terminal** in VS Code (click the + icon in the terminal panel or press **Ctrl+Shift+`**)

Before running the web client template tool, configure npm to use the NVIDIA package registry:

- Set up the NVIDIA npm registry:

```
npm config set "@nvidia:registry"
"https://edge.urm.nvidia.com/artifactory/api/npm/omniverse-client-npm/" --
location=user
```

Now run the web client template tool to scaffold a streaming client:

- Run the web client template tool:

```
npx @nvidia/create-ov-web-rtc-app
```

The tool will prompt you with several options. Follow these selections:

- **Project name** – Type `nvcf-stream` and press **Enter**
- **Author** – Accept the default or type your name
- **Project description** – Press **Enter** to accept the default
- **Output directory** – Press **Enter** to accept the default

- **Select UI framework** – Select **None TypeScript only**
- **Select stream source** – Select **NVCF (NVIDIA Cloud Functions)**
- **NVCF API endpoint** – Enter `http://localhost:20081`
- **API key** – Paste the API key from your `function-info.txt` file (starts with `nvapi-`)
- **Function ID** – Paste the function ID from your `function-info.txt` file
- **Streaming library** – Press **Enter** to accept the default

Tip

If you've lost your function ID, you can retrieve it by running:

```
nvcf function list
```

If you've lost your API key, generate a new one:

```
nvcf api-key generate
```

Remember to save the new key in `function-info.txt` – it can't be retrieved once the terminal output scrolls past.

Configure the Web Client

The NVCF web client generates a `frontend/` and `backend/` directory structure. The backend acts as an intermediate proxy that communicates with the NVCF API.

- Navigate into the newly created NVCF web client directory:

```
cd nvcf-stream
```
- Open `frontend/src/main.ts` and find the `signalingServer` configuration (around line 76)
- Replace `localhost` with **your instance's IP address** for `signalingServer`
- Find the `backendUrl` configuration and replace `localhost` with **your instance's IP address**

Note

In NVCF mode, you do **not** need to set a media server address. NVCF handles the UDP media routing automatically. The `backendUrl` points to the local backend proxy (which you'll start in the next section), not a media server.

- Save the file

Tip

Where do I find my IP address? Same as before – look at the URL in your browser's address bar. The IP address is the part before the colon and port number.

Install Dependencies and Start the Client

The NVCF web client runs as two separate processes – a **backend proxy** that communicates with the NVCF API, and a **frontend** that renders the stream in the browser. We'll start each one independently so you can see how they work together.

- Install the project dependencies from the `nvcf-stream` root directory:

```
npm install
```

Start the Backend Proxy

- Navigate into the backend directory:

```
cd backend
```

- Start the backend proxy:

```
npm run dev -- --host 0.0.0.0
```

Note

The backend proxy relays requests between your frontend and the NVCF API. It handles authentication and session management using the API key and function ID you provided during setup.

Start the Frontend

- **Open a new terminal tab** in VS Code (click the + icon or press **Ctrl+Shift+`**)

- Navigate into the frontend directory:

```
cd nvcf-stream/frontend
```

- Start the frontend development server:

```
npm run dev -- --host 0.0.0.0
```

Verify Cloud Streaming

- **Open a new terminal** in VS Code (click the + icon in the terminal panel or press **Ctrl+Shift+`**) and run the following command to get the URL for your stream:

```
echo "http://$(curl icanhazip.com):5173"
```

- **Control+click** the URL in the terminal to open it in your browser
- You should see the Data Center digital twin scene streaming from the cloud-deployed Kit application

Tip

If you see **“Waiting for stream...”** for more than 30 seconds, verify:

- The backend proxy terminal is still running (no errors)
- The frontend terminal is still running

- Your `signalingServer` and `backendUrl` in `frontend/src/main.ts` both use your instance's IP address (not `localhost`)
- Your NVCF function pod is in **Running (2/2)** state (`kubectl get pods -n nvcf-backend`)

Lab Cleanup

GPU resources in this lab environment are limited. Before wrapping up, let's clean up the running processes and cloud resources.

Stop Running Processes

Go to each terminal where a process is still running and press **Ctrl+C** to stop it:

- **Frontend terminal** (`npm run dev` in `frontend/`): Press **Ctrl+C**
- **Backend proxy terminal** (`npm run dev` in `backend/`): Press **Ctrl+C**
- Any other active terminals: Press **Ctrl+C**

Remove the NVCF Deployment

- Undeploy your function to free up the GPU node:

```
nvcf function deploy remove --function-id <functionId> --version-id <versionId>
```

Replace `<functionId>` and `<versionId>` with the values from your `function-info.txt` file.

Delete the API Key

- List your API keys:

```
nvcf api-key list
```
- Find the key ID from the output, then delete it:

```
nvcf api-key delete --key-id <KEY_ID>
```

Note

Cleaning up resources is an important habit in production environments. NVCF clusters have a limited number of GPU nodes available. By removing your deployment, you free up resources for other users and avoid unnecessary costs.

What's Next

Proceed to the [Conclusion](#) to review what you've learned and explore next steps for continuing your Kit App Streaming journey.

Conclusion

Congratulations on completing the lab. Let's take a moment to review the journey you've taken and reflect on the key concepts you've learned.

The Journey

You started with an empty workspace and, step by step, built a complete Kit App Streaming pipeline:

1. Built a Kit Application

Used the Kit App Template to scaffold a **USD Viewer** application with streaming and NVCF application layers. Built it from source using `repo.sh`.

2. Streamed Locally

Created a web client using `create-ov-web-rtc-app`, configured WebRTC connection parameters, and verified **direct local streaming** between the Kit app and a browser-based client.

3. Containerized for the Cloud

Packaged the Kit application into a **Docker container** and pushed it to a container registry, preparing it for deployment in a managed cloud environment.

4. Deployed to NVCF

Used the **NVCF CLI** to register the application as a function, create a deployment specification, and deploy it to a GPU node in the cloud.

5. Streamed from the Cloud

Created a second web client configured for **NVCF cloud streaming**, connected to the deployed Kit application, and verified end-to-end streaming from cloud GPU to browser.

Key Concepts

Here's a summary of the most important concepts you worked with today:

Concept	What It Is
Kit App Template	A template tool that generates buildable Kit applications with templates for common use cases
Application Layers	<code>.kit</code> configuration files that customize how an application behaves in different environments (local, streaming, NVCF)
WebRTC	The streaming protocol used to send GPU-rendered video from the Kit app to a web browser in real time
Signaling Server	A server that helps the Kit app and web client establish a WebRTC

Concept	What It Is
	connection
NVCF (NVIDIA Cloud Functions)	A managed platform for deploying and running GPU workloads in the cloud
Function Definition	A JSON specification (<code>create-function.json</code>) that tells NVCF what container to run and how to configure it
Function ID / Version ID	Unique identifiers used to reference a registered function and its specific version in NVCF
<code>kubectl</code>	The Kubernetes command-line tool for inspecting and managing cluster resources

What Happens in Production

In a real-world deployment, the workflow you've completed today extends further:

1. **Web hosting** – Your web client would be deployed to a web hosting service (like AWS S3 + CloudFront, Azure Static Web Apps, or a CDN) so end users can access it from anywhere
2. **Scaling** – NVCF can automatically scale the number of GPU instances based on demand, supporting hundreds or thousands of concurrent users
3. **Custom applications** – Instead of the default USD Viewer, you'd build custom Kit applications with tailored UI, business logic, and domain-specific extensions
4. **Security** – Production deployments add authentication, encryption (TLS), and access controls to protect streams and data

Continue Your Journey

Follow-On Lab: Develop AI-Powered Operational Dashboards

Ready to take the next step? The follow-on lab, **Develop AI-Powered Operational Dashboards for Digital Twin Applications**, picks up where this lab leaves off. You'll learn how to implement **custom bi-directional messaging** between your Kit application and web client, integrate live data sources, and build interactive operational dashboards on top of the streaming foundation you've created here.

Next Steps

To continue your Kit App Streaming journey, explore these resources:

NVIDIA Kit App Streaming

- [Kit App Streaming Documentation](#) – Comprehensive guides for architecture, deployment, and configuration

- [Kit App Streaming Collection on NGC](#) – Container images, Helm charts, and deployment resources
- [Web Development Guide](#) – Detailed guidance on embedding Kit streams into web applications

Kit Application Development

- [Kit App Template Repository](#) – The template tool used in this lab, with additional templates and examples
- [Kit App Template Companion Tutorial](#) – Deep dive into building Kit applications with the Kit SDK
- [Omniverse Kit App Development Guide](#) – Comprehensive documentation for Kit application development

Web Client and Streaming Libraries

- [WebRTC Streaming Library](#) – Documentation for the NVIDIA Omniverse WebRTC Streaming Library used by web clients
- [Embedded Web Viewer](#) – Ready-to-use embeddable web viewer for Kit applications
- [Web Viewer Sample](#) – Reference implementation for building custom web clients

Digital Twins and OpenUSD

- [OpenUSD Learning Path](#) – Comprehensive learning path for building digital twins with OpenUSD
- [Connect Real-Time IoT Data to Digital Twins](#) – Tech blog on integrating live IoT data with 3D digital twins

Tip

If you'd like to explore further while you still have access to this lab environment, try these exercises:

- Run `kubectl get pods -A` and examine the full list of services that make up the Kit App Streaming platform
- Explore the Kit application source code in `kit-app-template/source/` to understand the extension structure
- Look at the `.kit` files in the application to see how streaming is configured
- Modify the web client's HTML/CSS to customize the streaming interface

Thank You

Thank you for participating in this lab. We hope you've gained a solid understanding of the Kit App Streaming development and deployment lifecycle. The skills you've practiced today – building Kit apps, configuring web clients, containerizing applications, and deploying to NVCF – are directly applicable to production digital twin projects.

We'd love your feedback. Please take a moment to fill out the survey linked in the navigation bar.